

Performer: the Kernel Trick & Associativity.

Softmax attention is $O(T^2)$ for one stubborn reason: the softmax *couples* every query–key pair, so the $T \times T$ score matrix must be built *before* it can touch V . Performer breaks the coupling with a kernel feature map ϕ , turning $\exp(qk)$ into an *inner product* $\phi(q)\phi(k)$. Once it factorises, matrix multiplication’s associativity lets us compute $K^\top V$ *first* — collapsing $(QK^\top)V$ into $Q(K^\top V)$ and the cost from $O(T^2)$ to $O(T)$. Every number below is computed in full, nothing summarised or deferred.

THE EQUATION

$$\exp(q \cdot k) \approx \phi(q) \cdot \phi(k) \quad \implies \quad (QK^\top)V = Q(K^\top V)$$

Worked example. $T=4$, $d=2$, explicit feature map $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$. We compute attention *both* ways — $(\phi(Q)\phi(K)^\top)V$ and $\phi(Q)(\phi(K)^\top V)$ — show they are bit-for-bit identical, and count 80 vs. 48 multiplies to expose the $O(T^2)$ vs. $O(T)$ gap.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Why softmax attention is stuck at $O(T^2)$	2
2	The kernel trick: factorise the weight	2
3	Associativity: the same product, reordered	3
4	Worked computation ($T=4$, $d=2$, $m=3$)	4
5	Performer in practice: FAVOR+	5
6	Properties / consequences that matter	5
7	Where this sits in the track	6
A	Reproducible (NumPy)	6

1 Why softmax attention is stuck at $O(T^2)$

Write attention row by row. For query i , the (unnormalised) output is

$$o_i = \sum_{j=1}^T \frac{\exp(q_i \cdot k_j)}{\sum_{\ell} \exp(q_i \cdot k_{\ell})} v_j.$$

The numerator weight $\exp(q_i \cdot k_j)$ is a *nonlinear, non-separable* function of the *pair* (i, j) : there is no way to write $\exp(q_i \cdot k_j)$ as $a_i b_j$. So we must evaluate all T^2 exponentials, store them as the score matrix P , normalise each row, and only then multiply by V . Forming $QK^{\top}$ is $O(T^2d)$ and the product PV is another $O(T^2d)$.

A tempting “optimisation” is to reorder $(QK^{\top})V$ into $Q(K^{\top}V)$ to avoid the big matrix. It is *invalid* for softmax. The softmax sits *between* $QK^{\top}$ and V : $\text{softmax}(QK^{\top})V$. The row-wise exponential-and-normalise is a nonlinear operator $\mathcal{N}(\cdot)$, and matrix associativity only applies to plain products: $\mathcal{N}(QK^{\top})V \neq Q\mathcal{N}(K^{\top})V$. You cannot push V inside a nonlinearity. The $T \times T$ matrix is mandatory — *as long as the weight is exp of a dot product*.

2 The kernel trick: factorise the weight

Suppose we can find a feature map $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that the attention weight *factorises*:

$$\exp(q_i \cdot k_j) \approx \phi(q_i) \cdot \phi(k_j) = \sum_{r=1}^m \phi(q_i)_r \phi(k_j)_r. \quad (1)$$

Now the weight *is* separable into a query part and a key part. Substitute into the numerator of the output and exchange the order of summation:

$$\sum_j (\phi(q_i) \cdot \phi(k_j)) v_j = \sum_j \sum_r \phi(q_i)_r \phi(k_j)_r v_j = \sum_r \phi(q_i)_r \underbrace{\sum_j \phi(k_j)_r v_j}_{\text{independent of } i} = \phi(q_i) \cdot \underbrace{\left(\sum_j \phi(k_j) v_j^\top \right)}_{= KV \in \mathbb{R}^{m \times d}}.$$

The inner sum $KV = \sum_j \phi(k_j) v_j^\top$ does *not* depend on which query i we are at, so we compute it *once* and reuse it for all T queries. The same trick handles the denominator with $v_j = 1$: let $z = \sum_j \phi(k_j) \in \mathbb{R}^m$, then the normaliser for query i is $\phi(q_i) \cdot z$.

$$o_i = \frac{\phi(q_i) (K^\top V)}{\phi(q_i) \cdot z}, \quad KV = \sum_j \phi(k_j) v_j^\top \in \mathbb{R}^{m \times d}, \quad z = \sum_j \phi(k_j) \in \mathbb{R}^m. \quad (2)$$

The softmax forces you to first compare *every* query with *every* key, then mix in the values — an $O(T^2)$ all-pairs handshake. The kernel map lets each key “deposit” its value into a single shared $m \times d$ accumulator KV *before* any query arrives. Every query then makes one cheap read from that accumulator. Nobody compares pairs directly; the keys summarise themselves into a fixed-size memory, and queries just look things up.

3 Associativity: the same product, reordered

Stack ϕ over the sequence: $\Phi_Q = \phi(Q) \in \mathbb{R}^{T \times m}$, $\Phi_K = \phi(K) \in \mathbb{R}^{T \times m}$. The factorised attention numerator is the plain matrix product $\Phi_Q \Phi_K^\top V$ — and now there is *no nonlinearity in between*, so associativity is legal:

$$(\Phi_Q \Phi_K^\top) V = \Phi_Q (\Phi_K^\top V) \quad (3)$$

Left order builds the $T \times T$ matrix $\Phi_Q \Phi_K^\top$ first: $O(T^2 m) + O(T^2 d)$. **Right order** builds the $m \times d$ matrix $\Phi_K^\top V$ first: $O(T m d) + O(T m d) = O(T m d)$. With m fixed this is $O(T)$. The speed-up is $T^2(m+d)/(2T m d) = \frac{T(m+d)}{2 m d} = O(T)$.

4 Worked computation ($T=4$, $d=2$, $m=3$)

Inputs and an explicit feature map $\phi([x_1, x_2]) = [1, x_1, x_2] \in \mathbb{R}^3$ (a deterministic stand-in for FAVOR+ random features — it is separable, which is all the algebra needs):

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \\ 2 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & 1 \end{bmatrix}.$$

Apply ϕ row-wise (prepend a 1):

$$\Phi_Q = \phi(Q) = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & -1 \end{bmatrix}, \quad \Phi_K = \phi(K) = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 2 & 0 \end{bmatrix} \in \mathbb{R}^{4 \times 3}.$$

4.1 Order A — $(\Phi_Q \Phi_K^\top) V$, the $O(T^2)$ path

First the full 4×4 score matrix $S = \Phi_Q \Phi_K^\top$. Entry (1, 1) is $[1, 1, 0] \cdot [1, 1, 1] = 2$; entry (1, 4) is $[1, 1, 0] \cdot [1, 2, 0] = 3$:

$$S = \Phi_Q \Phi_K^\top = \begin{bmatrix} 2 & 2 & 1 & 3 \\ 2 & 1 & 2 & 1 \\ 3 & 2 & 2 & 3 \\ 1 & 2 & 0 & 3 \end{bmatrix} \in \mathbb{R}^{4 \times 4}.$$

Then $O_A = S V$. Row 1: $2[1, 0] + 2[0, 1] + 1[1, 1] + 3[2, 1] = [9, 6]$:

$$O_A = S V = \begin{bmatrix} 9 & 6 \\ 6 & 4 \\ 11 & 7 \\ 7 & 5 \end{bmatrix} \in \mathbb{R}^{4 \times 2}.$$

4.2 Order B — $\Phi_Q(\Phi_K^\top V)$, the $O(T)$ path

First the small 3×2 accumulator $KV = \Phi_K^\top V$. Row 1 (the constant feature) sums all values: $[1, 0] + [0, 1] + [1, 1] + [2, 1] = [4, 3]$. Row 2 weights values by each key's x_1 : $1[1, 0] + 1[0, 1] + 0[1, 1] + 2[2, 1] = [5, 3]$. Row 3 by x_2 : $1[1, 0] + 0 + 1[1, 1] + 0 = [2, 1]$:

$$KV = \Phi_K^\top V = \begin{bmatrix} 4 & 3 \\ 5 & 3 \\ 2 & 1 \end{bmatrix} \in \mathbb{R}^{3 \times 2}.$$

Then $O_B = \Phi_Q(KV)$. Row 1: $[1, 1, 0]$ columns = $[1 \cdot 4 + 1 \cdot 5 + 0, 1 \cdot 3 + 1 \cdot 3 + 0] = [9, 6]$:

$$O_B = \Phi_Q(KV) = \begin{bmatrix} 9 & 6 \\ 6 & 4 \\ 11 & 7 \\ 7 & 5 \end{bmatrix} \in \mathbb{R}^{4 \times 2}.$$

4.3 They are identical

$O_A = O_B$ exactly — maximum absolute difference 0. Associativity is an *equality*, not an approximation; reordering changes only the *cost*, never the result.

$(\Phi_Q \Phi_K^\top)V = \Phi_Q(\Phi_K^\top V)$ is just the associativity of matrix multiplication. The entire $O(T^2) \rightarrow O(T)$ win is: *multiply the two small things first*. The kernel map ϕ is the enabler — it removes the softmax nonlinearity that otherwise sits between $QK^\top$ and V and forbids the reorder.

4.4 Counting the multiplications

With $T=4, m=3, d=2$:

Order	operations	multiplies
A ($O(T^2)$)	$\Phi_Q \Phi_K^\top$ ($T^2 m = 48$) + $S V$ ($T^2 d = 32$)	80
B ($O(T)$)	$\Phi_K^\top V$ ($T m d = 24$) + $\Phi_Q(KV)$ ($T m d = 24$)	48

Even at $T = 4$ the reorder is cheaper. The gap explodes with T (quadratic $T^2(m+d)$ vs. linear $2Tmd$), shaded below (intensity $\propto$ the quadratic cost, capped at **indigo!80**):

	$T=4$	64	256	1024	4096
quad / lin ratio	1.7	26.7	106.7	426.7	1706.7

4.5 Normalisation, made explicit

The denominator uses $z = \Phi_K^\top \mathbf{1} = \sum_j \phi(k_j) = [4, 4, 2]$. Per query $d_i = \phi(q_i) \cdot z$: $d = [8, 6, 10, 6]$. Dividing O_B row-wise gives the proper attention output $\text{attn}_i = o_i/d_i$:

$$\text{attn} = \begin{bmatrix} 1.125 & 0.75 \\ 1 & 0.6667 \\ 1.1 & 0.7 \\ 1.1667 & 0.8333 \end{bmatrix}.$$

The normaliser rides along the *same* linear path — one extra length- m accumulator z — so the whole attention stays $O(T)$.

5 Performer in practice: FAVOR+

Our $\phi = [1, x_1, x_2]$ is a toy. Performer's **FAVOR+** (Fast Attention Via positive Orthogonal Random features) chooses a *random* feature map of width m , $\phi(x) \propto \exp(\omega_r^\top x - \frac{\|x\|^2}{2})$ with ω_r drawn from a Gaussian (and orthogonalised), so that $\mathbb{E}[\phi(q) \cdot \phi(k)] = \exp(q \cdot k)$ *exactly*. The estimate of the softmax (Gaussian) kernel is therefore *unbiased*; the *positive* random features keep all entries ≥ 0 (so the implied weights and normaliser stay valid) and sharply lower the variance. Crucially, m is a fixed hyperparameter independent of T , so the $O(Tmd)$ cost holds for *any* length, and the accumulator form supports causal decoding via a running prefix-sum of KV .

6 Properties / consequences that matter

1. **Linear time and memory.** $O(Tmd)$ time, $O(md + Td)$ memory — no $T \times T$ matrix is ever stored, removing the quadratic activation-memory wall.

2. **Length-flexible.** m is fixed; the same weights run at any T , and the prefix-sum form makes it *causal-friendly* (a strength over vanilla Linformer's fixed- T projections).
3. **Unbiased kernel estimate.** FAVOR+ gives $\mathbb{E}[\phi(q)\cdot\phi(k)] = \exp(q\cdot k)$; more features m lower variance at a linear cost increase.
4. **Associativity is exact.** The reorder $(QK^\top)V = Q(K^\top V)$ is an identity once the weight is factorised; only the kernel approximation (1) introduces error, not the reorder (3).

Performer is an *approximation*: $\phi(q)\cdot\phi(k)$ only equals $\exp(q\cdot k)$ in *expectation*, with random-feature variance $\sim 1/m$. Average LM loss is fairly robust, but *precise retrieval* (copying, exact long-range lookup) degrades more — the noise blurs the sharp, near-one-hot attention those tasks rely on. Raising m helps but eats into the speed-up, and too few features can even make the estimated denominator small or unstable.

7 Where this sits in the track

This closes Module 5.3 by contrasting the two routes to linear attention:

	Linformer (Topic 1)	Performer (Topic 2)
mechanism	low-rank length projection E, F	kernel feature map ϕ + reorder
cost	$O(Tkd)$ (fixed k)	$O(Tmd)$ (fixed m)
nature	deterministic, learned	stochastic random features (unbiased)
length	assumes fixed T	length-flexible, causal-friendly
main weakness	fixed- k info bottleneck	random-feature variance

Both deliver $O(T)$ attention; Phase 5 continues with sparse and state-space alternatives that attack the same quadratic wall from yet other angles.

A Reproducible (NumPy)

Inputs: $Q, K, V \in \mathbb{R}^{4 \times 2}$ above and $\phi(x) = [1, x_1, x_2]$. The snippet builds both orderings, checks they match exactly, prints the multiply counts, and the normalised attention.

```
import numpy as np
Q=np.array([[1,0],[0,1],[1,1],[1,-1]],float)
K=np.array([[1,1],[1,0],[0,1],[2,0]],float)
V=np.array([[1,0],[0,1],[1,1],[2,1]],float)
phi=lambda X: np.column_stack([np.ones(len(X)),X[:,0],X[:,1]])
PQ,PK=phi(Q),phi(K) # (4,3) each
T,m,d=4,3,2
OA=(PQ@PK.T)@V # order A: T x T first
OB=PQ@(PK.T@V) # order B: m x d first
print(OA); print(OB); print(np.abs(OA-OB).max()) # ... 0.0
print("multA", T*T*(m+d), "multB", 2*T*m*d) # 80 48
z=PK.sum(0); den=PQ@z; attn=(PQ@(PK.T@V))/den[:,None]
```

```
print(np.round(attn,4))
# KV = PK.T@V = [[4,3],[5,3],[2,1]]; OA row1 = [9,6]
# attn row1 = [1.125 0.75]
```

Marevlo Research · Transformer Track · Module 5.3 · Topic 2 of 2.